UNIVERSIDAD TECNOLÓGICA PRIVADA DE SANTA CRUZ

FACULTAD DE CIENCIAS & TECNOLOGÍA

INGENIERÍA DE SISTEMAS

ALGORITMOS Y PROGRAMACION

PROYECTO FINAL:

"SISTEMA DE GESTIÓN DE CLIENTES, SERVICIOS Y CITAS PARA LA BARBERÍA ZURDO"

DOCENTE: **ING. PAUL MAURICIO MELGAR ZABALA**

INTEGRANTES: **CHARLES LUIS CEDEÑO CANIDO**

LUIS ANGEL VELASQUEZ ROMAN

JOSE MIGUEL CAZAS VIDAURRE

OSCAR PEREZ MONTEMURRO

SANTA CRUZ DE LA SIERRA – BOLIVIA

26/10/2026

ÍNDICE

1. INTRODUCCIÓN 1

2. PLANTEAMIENTO DEL PROBLEMA 1

2.1. ALCANCE DEL SISTEMA 2

2.2. LIMITACIONES 3

3. OBJETIVOS 3

3.1. OBJETIVO GENERAL 3

3.2. OBJETIVOS ESPECÍFICOS 3

4. DESCRIPCIÓN Y FUNCIONALIDADES DEL SISTEMA 4

4.1. CONTEXTO DE LA BARBERÍA 4

4.2. ESTRUCTURA DE LA APLICACIÓN 5

4.3. MÓDULO DE BARBEROS 5

4.4. MÓDULO DE CLIENTES 6

4.5. MÓDULO DE CITAS 9

4.6. MÓDULO DE ARCHIVOS 12

4.7. CUMPLIMIENTO DE LAS FUNCIONALIDADES MÍNIMAS 13

5. DIAGRAMA UML DE CLASES 13

5.1. CLASES IDENTIFICADAS 14

5.2. RELACIONES ENTRE CLASES 14

5.3. DIAGRAMA 15

6. DESCRIPCIÓN TÉCNICA Y PROGRAMACIÓN 16

6.1. ORGANIZACIÓN DEL CÓDIGO 16

6.2. UNIDAD 1: CODIFICACIÓN DE ALGORITMOS 16

6.3. UNIDAD 2: TIPOS DE DATOS COMPUESTOS 18

6.4. UNIDAD 3: ARCHIVOS DE TEXTO 19
6.5. UNIDAD 4: PROGRAMACIÓN ORIENTADA A OBJETOS 20
6.5.1. Encapsulamiento 20
6.5.2. Abstracción 20
6.5.3. Herencia y polimorfismo 22
6.5.4. Miembros estáticos y métodos de instancia 22
6.6. DESCRIPCIÓN DE CLASES, ATRIBUTOS Y MÉTODOS 22
6.7. UNIDAD 5: VALIDACIÓN DE APLICACIONES 24
6.8. OPERACIONES DE NEGOCIO 27
7. PRUEBAS Y RESULTADOS 27
7.1. ENTORNO Y METODOLOGÍA 27
7.2. PRUEBAS DE CLIENTES 28
7.3. PRUEBAS DE CITAS 29
7.4. PRUEBAS DE ARCHIVOS 30
7.5. RESUMEN DE RESULTADOS 31
8. CONCLUSIONES 32
9. BIBLIOGRAFÍA Y FUENTES CONSULTADAS 32
10. INVESTIGACIÓN E INNOVACIÓN 33
10.1. NECESIDAD ADICIONAL QUE RESUELVE 33
10.2. DESCRIPCIÓN DE LA MEJORA 33
10.3. FUENTES CONSULTADAS 34
10.4. ALTERNATIVAS CONSIDERADAS 34
10.5. JUSTIFICACIÓN DE LA ELECCIÓN 34
10.6. IMPLEMENTACIÓN 34
10.7. EVIDENCIAS DE FUNCIONAMIENTO 36

ANEXOS 36
ANEXO A: INSTRUCCIONES DE EJECUCIÓN 36
ANEXO B: ARCHIVO DE DATOS DE EJEMPLO 37
ANEXO C: CÓDIGO DE LAS CLASES 38

1. INTRODUCCIÓN

En la actualidad, los negocios de servicios personales, como las barberías, dependen en gran medida de una buena organización de su agenda para atender con calidad a sus clientes y aprovechar mejor el tiempo de su personal. Cuando las citas se anotan en cuadernos, en mensajes sueltos o simplemente de memoria, es muy común que aparezcan cruces de horarios, olvidos, errores al calcular los precios y pérdida de la información de los clientes.

Frente a esta situación, el presente proyecto desarrolla una aplicación informática funcional para la Barbería Zurdo, ubicada sobre la Av. San Martín, C. Fermin Peralta, en la ciudad de Santa Cruz de la Sierra, con atención de 9:00 a 23:00. El sistema permite registrar clientes, consultar a los barberos y los servicios que ofrece la barbería, reservar citas verificando que el barbero esté libre en la fecha y hora elegidas, calcular automáticamente el total a pagar con descuento para clientes VIP, y guardar y recuperar toda la información mediante archivos.

La aplicación fue construida con HTML y CSS para la interfaz y JavaScript como lenguaje principal, sin utilizar base de datos ni frameworks. El código aplica programación orientada a objetos (una clase abstracta *Persona*, las clases derivadas *Cliente* y *Barbero*, y las clases *Servicio* y *Cita*), arreglos de objetos, operaciones sobre cadenas de texto, archivos en formato JSON y CSV, validaciones en HTML y JavaScript, y manejo de errores con *try*, *catch*, *throw* y *finally*. De esta manera se cubren los contenidos de las cinco unidades de la asignatura.

Este documento técnico presenta el planteamiento del problema y los objetivos, la descripción de las funcionalidades del sistema con capturas de pantalla, la descripción técnica de las clases y de la programación, las pruebas realizadas y sus resultados, las conclusiones, las fuentes consultadas y, el anexo de investigación e innovación que documenta las mejoras adicionales implementadas por el equipo.

2. PLANTEAMIENTO DEL PROBLEMA

En una barbería, cada atención involucra cuatro elementos que deben coordinarse correctamente: el cliente, el barbero, el servicio que se va a realizar y el horario en que se atenderá. Cuando esta coordinación se hace de forma manual, sin un sistema que apoye al negocio, aparecen dificultades que afectan tanto a los clientes como a quienes administran la barbería. Entre las principales se encuentran las siguientes:

- Cruces de horario: dos clientes pueden quedar agendados con el mismo barbero a la misma hora, porque nadie verifica la disponibilidad antes de confirmar.
- Datos de clientes incompletos o incorrectos: nombres escritos de distintas formas, teléfonos con menos dígitos de los necesarios o registrados más de una vez.
- Cálculo manual de precios: el descuento para clientes frecuentes (VIP) se calcula a mano, con riesgo de errores y de discusiones al momento de cobrar.
- Dificultad para buscar información: encontrar a un cliente o una cita específica en un cuaderno o en una conversación de mensajes toma tiempo.
- Falta de control del estado de las citas: no se distingue con claridad qué citas están pendientes, cuáles se completaron y cuáles se cancelaron, ni cuánto dinero se cobró realmente.
- Pérdida de información: si el cuaderno se extravía o el teléfono se daña, se pierde el historial de clientes y de citas.

Por lo tanto, la barbería necesita un sistema sencillo, que pueda abrirse en cualquier computadora sin instalaciones complicadas, que impida registrar datos inválidos, que evite que un barbero tenga dos citas a la misma hora, que calcule automáticamente los totales y que permita guardar y recuperar la información en archivos. A partir de esta necesidad se plantea la siguiente pregunta:

¿Cómo desarrollar una aplicación web sencilla, ejecutable sin servidor ni base de datos, que permita gestionar los clientes, servicios, horarios y citas de la Barbería Zurdo, evitando cruces de horario, validando los datos ingresados y conservando la información entre una ejecución y otra?

2.1. ALCANCE DEL SISTEMA

El sistema desarrollado abarca la gestión de clientes (registro, listado, búsqueda, modificación y eliminación), la consulta de los barberos y de los servicios con sus precios, la reserva y administración de citas (registro, modificación, cambio de estado y eliminación), el cálculo de totales con descuento VIP, el resumen de ingresos cobrados, y el guardado, exportación e importación de los datos mediante archivos JSON y CSV.

2.2. LIMITACIONES

Por tratarse de un proyecto académico con tiempo limitado, el sistema tiene las siguientes limitaciones: el catálogo de barberos y de servicios está definido dentro del código y no se edita desde la pantalla; no cuenta con inicio de sesión ni roles de usuario; la información se almacena en el navegador de la computadora donde se usa, por lo que es un sistema de un solo equipo; y las citas se reservan en bloques de una hora, dentro del horario de atención de 9:00 a 23:00 (la última cita puede reservarse a las 22:00).

3. OBJETIVOS

3.1. OBJETIVO GENERAL

Desarrollar una aplicación web funcional, utilizando HTML, CSS y JavaScript, que permita gestionar los clientes, servicios, horarios y citas de la Barbería Zurdo, aplicando los contenidos de las cinco unidades de la materia.

3.2. OBJETIVOS ESPECÍFICOS

- Analizar el proceso de atención de la barbería para identificar las entidades, los datos y las reglas del negocio que debe atender el sistema.
- Implementar las clases *Persona*, *Cliente*, *Barbero*, *Servicio* y *Cita* aplicando herencia, polimorfismo, encapsulamiento y abstracción.
- Administrar los registros mediante arreglos de objetos y aplicar operaciones sobre cadenas, como la normalización, la búsqueda y la comparación de texto.
- Registrar, listar, buscar, modificar y eliminar clientes y citas, incorporando reglas de negocio como la disponibilidad de los barberos, el descuento VIP y el cálculo del total.
- Validar los datos mediante atributos de HTML, funciones de JavaScript y eventos (*input*, *blur*, *change* y *submit*), manejando los errores con *try*, *catch*, *throw* y *finally*.
- Guardar y recuperar la información en archivos de texto (JSON) y exportar un reporte en formato CSV.
- Investigar e implementar mejoras que aporten valor real a la solución: reporte CSV, lectura de archivos con *FileReader*, filtros combinados y vista de referencia de cada servicio.

4. DESCRIPCIÓN Y FUNCIONALIDADES DEL SISTEMA

En esta sección se describe el sistema desde el punto de vista del usuario: qué datos maneja, cómo está organizado y qué puede hacer cada uno de sus módulos. Todas las capturas corresponden a la aplicación en funcionamiento.

4.1. CONTEXTO DE LA BARBERÍA

La Barbería Zurdo cuenta con cuatro barberos, cada uno con una especialidad, y ofrece ocho servicios con precio fijo en bolivianos (Bs). Estos datos son la base sobre la que se reservan las citas.

Tabla 1

Barberos de la Barbería Zurdo

Nombre	Teléfono	Especialidad
Luis Velasquez	77371879	Cortes clásicos
Charles Cedeño	74172238	Fade y degradados
Josep Cazas	76611394	Barba
Oscar Perez	75350290	Mohicano y mullet

Fuente: Elaboración propia.

Tabla 2

Servicios y precios

Servicio	Precio (Bs)	Descripción
Taper Fade	50	Degradado progresivo en los laterales.
High Fade	55	Degradado alto con laterales definidos.
Mid Fade	50	Degradado medio con acabado limpio.
Low Fade	45	Degradado bajo y discreto.
Mohicano	60	Estilo mohicano con laterales cortos.
Mullet	55	Corte mullet con mayor longitud en la nuca.
Barba	25	Arreglo y perfilado de barba.

Servicio	Precio (Bs)	Descripción
Tinte	115	Coloración del cabello.

Fuente: Elaboración propia.

4.2. ESTRUCTURA DE LA APLICACIÓN

La aplicación se entrega en una carpeta que se ejecuta abriendo el archivo HTML en un navegador, sin instalar nada y sin conexión a internet. Los archivos que la componen son los siguientes:

index.html: Estructura de la página: barberos, clientes, citas y archivos. Contiene los formularios con sus validaciones HTML.

estilos.css: Estilos visuales de la interfaz (colores, tablas, formularios, botones y estados de las citas).

script.js: Lógica de la aplicación: clases, validaciones, manejo de datos, archivos y eventos.

datosejemplo.json: Archivo de datos de ejemplo (3 clientes y 4 citas) para probar la importación.

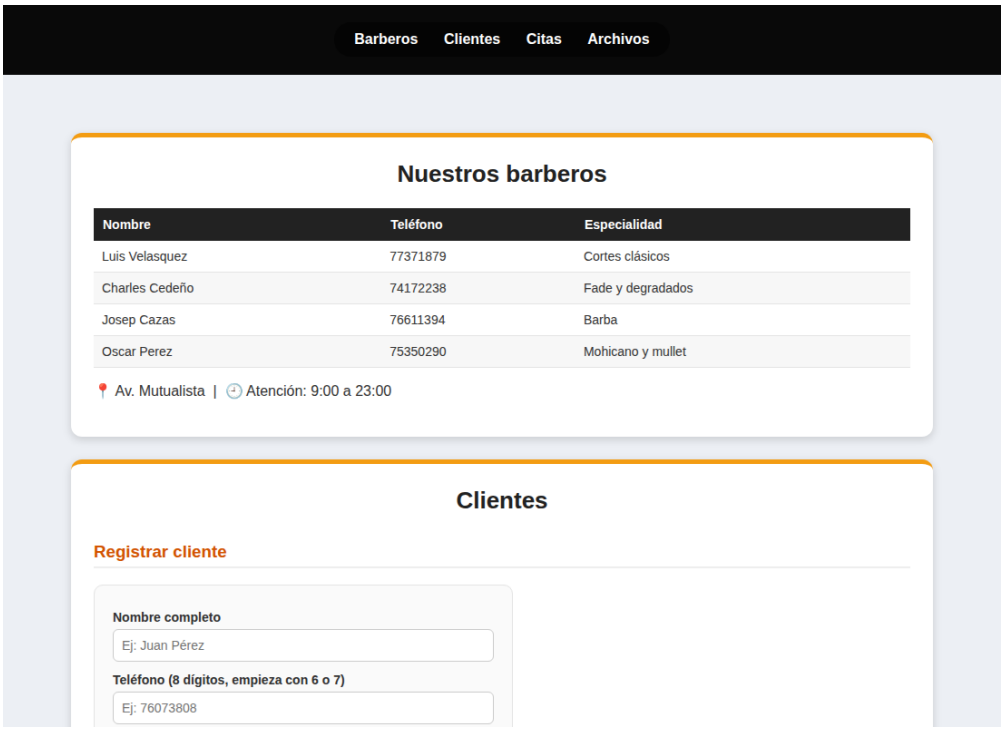
imagenes/: Siete imágenes de referencia de los servicios y de la barbería.

4.3. MÓDULO DE BARBEROS

Al abrir la aplicación se muestra el encabezado de la Barbería Zurdo con el menú de navegación (Barberos, Clientes, Citas y Archivos). Inmediatamente debajo aparece la sección Nuestros barberos, con la tabla de los barberos, sus teléfonos y sus especialidades, además de la dirección y el horario de atención de la barbería.

Figura 1

Pantalla principal de la aplicación con el menú de navegación



Fuente: Elaboración propia.

Figura 2

Sección «Nuestros barberos» con dirección y horario de atención



Fuente: Elaboración propia.

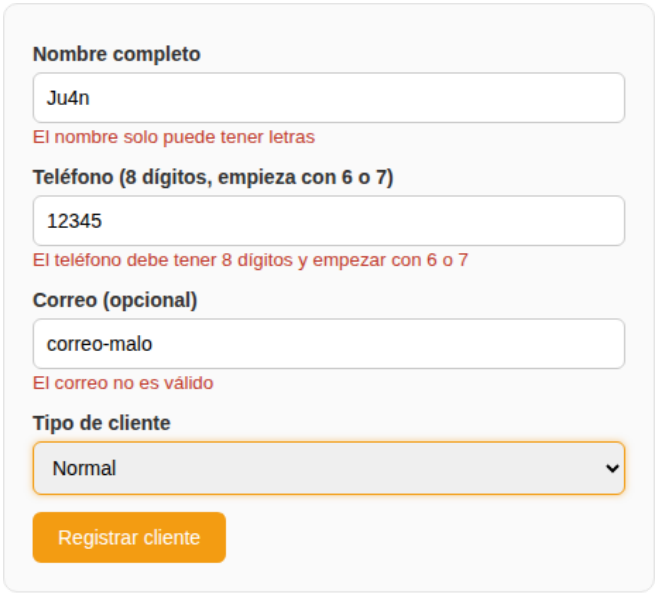
4.4. MÓDULO DE CLIENTES

Este módulo permite administrar a las personas que se atienden en la barbería. Sus funcionalidades son:

- Registro: el formulario solicita nombre completo, teléfono, correo y tipo de cliente (Normal o VIP con 15 % de descuento). El nombre se guarda con formato correcto: por ejemplo, juan PEREZ se guarda como Juan Perez.
- Validación en vivo: mientras la persona escribe, el sistema muestra debajo de cada campo un mensaje de error si el dato no cumple las reglas (véase la Figura 3).
- Teléfono único: no se permite registrar dos clientes con el mismo número de teléfono.
- Listado y búsqueda: la tabla muestra todos los clientes y el cuadro de búsqueda filtra por nombre o teléfono sin distinguir mayúsculas ni tildes.
- Modificación: el botón «Editar» carga los datos del cliente en el formulario para cambiarlos.
- Eliminación: el botón «Eliminar» borra al cliente, pero solo si no tiene citas registradas, para no dejar citas huérfanas.

Figura 3

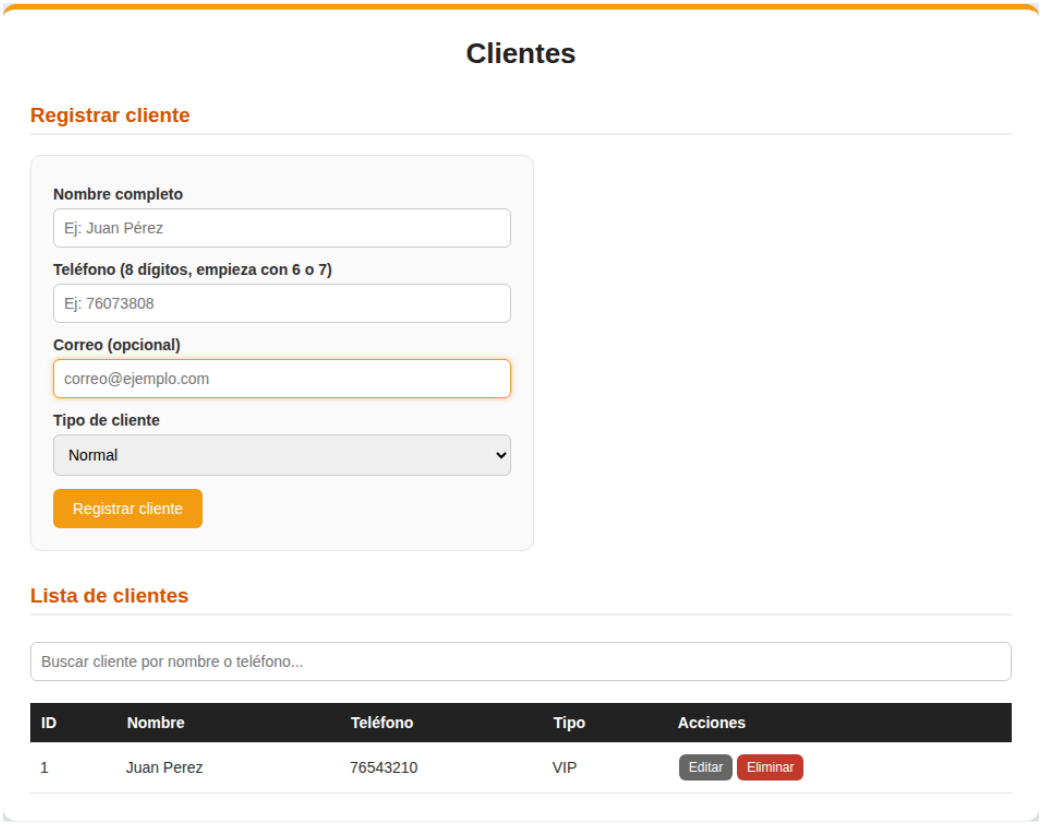
Validación en vivo del formulario de clientes con datos incorrectos



Fuente: Elaboración propia.

Figura 4

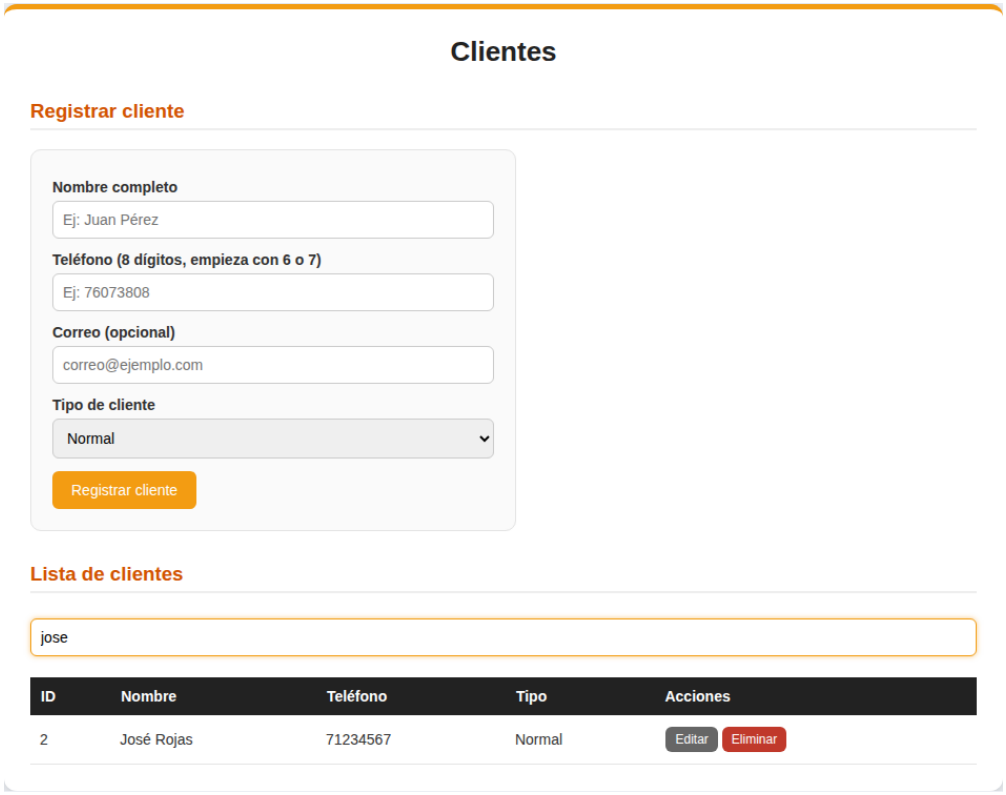
Cliente registrado correctamente y mostrado en la lista de clientes



Fuente: Elaboración propia.

Figura 5

Búsqueda de clientes: al escribir «jose» se encuentra a «José Rojas»



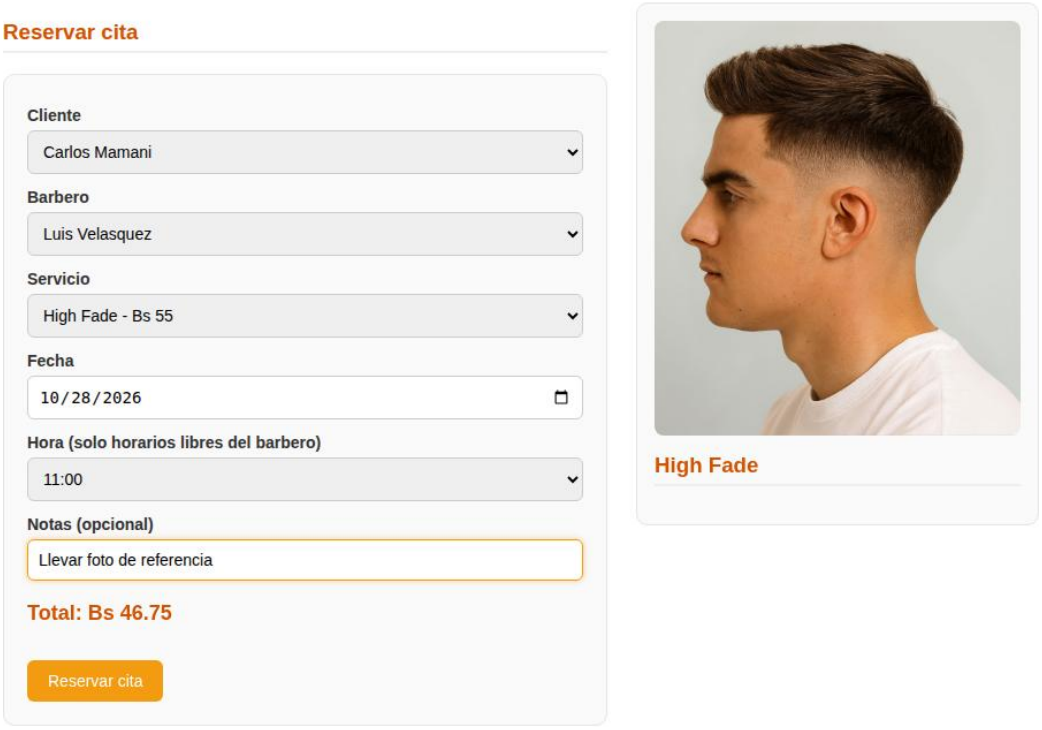
Fuente: Elaboración propia.

4.5. MÓDULO DE CITAS

Es el módulo principal del sistema. Para reservar una cita se elige el cliente, el barbero, el servicio y la fecha; luego el sistema muestra únicamente las horas libres de ese barbero en esa fecha. Al elegir el servicio aparece una imagen de referencia con su nombre, y el sistema calcula y muestra el total a pagar, aplicando el descuento del 15 % si el cliente es VIP.

Figura 6

Formulario de reserva con imagen de referencia del servicio y total con descuento VIP

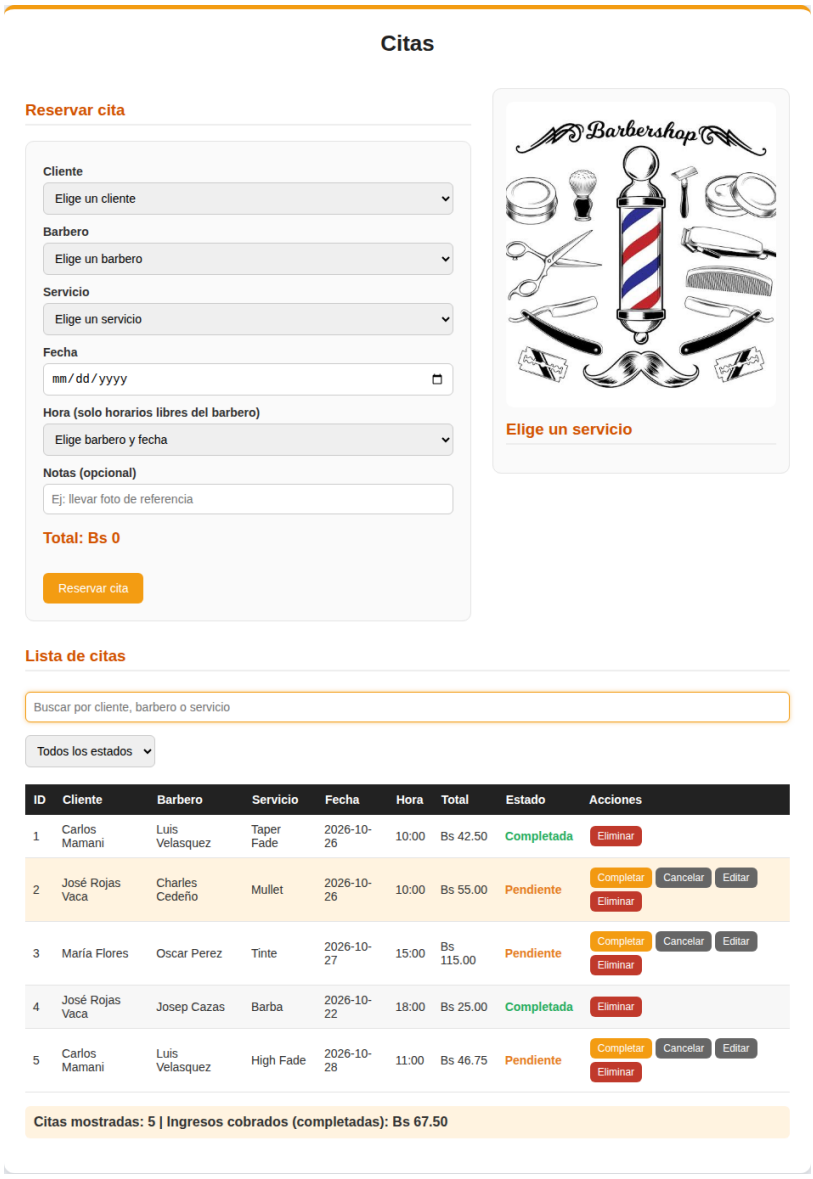


Fuente: Elaboración propia.

Las citas registradas se muestran en una tabla con su cliente, barbero, servicio, fecha, hora, total y estado (Pendiente, Completada o Cancelada). Desde las acciones de cada fila se puede completar, cancelar, editar o eliminar la cita; las citas ya completadas o canceladas solo pueden eliminarse. Debajo de la tabla se muestra un resumen con la cantidad de citas visibles y los ingresos cobrados, que suman únicamente las citas completadas.

Figura 7

Sección de citas con el formulario, la lista de citas y el resumen de ingresos



Fuente: Elaboración propia.

La lista admite filtros combinados: un cuadro de texto que busca por cliente, barbero o servicio, y un selector de estado. Ambos filtros actúan al mismo tiempo, y el resumen de ingresos se recalcula con las citas que quedan visibles.

Figura 8

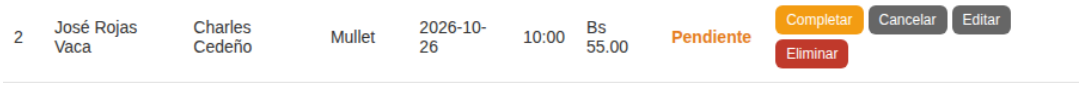
Lista de citas filtrada por estado Completada con ingresos cobrados de Bs 67.50



Fuente: Elaboración propia.

Figura 9

Lista de citas filtrada por texto: al escribir charles solo se muestra su cita



Fuente: Elaboración propia.

4.6. MÓDULO DE ARCHIVOS

Este módulo permite conservar y recuperar la información. El botón Exportar JSON descarga un archivo con todos los clientes y las citas; el botón Importar JSON lee un archivo guardado previamente y recupera los datos; y el botón Exportar reporte CSV genera una tabla de las citas que puede abrirse en Excel. Además, la aplicación guarda automáticamente una copia en el navegador (*localStorage*) después de cada cambio, de modo que los datos siguen disponibles al cerrar y volver a abrir la página.

Figura 10

Sección Guardar y recuperar datos con los botones de exportación e importación



Fuente: Elaboración propia.

4.7. CUMPLIMIENTO DE LAS FUNCIONALIDADES MÍNIMAS

A continuación se relaciona cada funcionalidad mínima exigida en la consigna con la forma en que fue resuelta en el sistema:

Registrar información de al menos dos entidades relacionadas: Se registran clientes y citas. Cada cita se relaciona con un cliente (por su identificador), con un barbero y con un servicio.

Listar registros y permitir buscar o filtrar: Lista de clientes con búsqueda por nombre o teléfono; lista de citas con búsqueda por texto y filtro por estado.

Modificar o eliminar registros: Edición y eliminación de clientes; edición, cancelación, finalización y eliminación de citas.

Realizar una operación de negocio: Control de disponibilidad del barbero, cálculo del total con descuento VIP y suma de ingresos cobrados.

Mostrar mensajes claros de éxito y error; impedir datos inválidos: Mensajes en verde (éxito) y en rojo (error) en la parte superior, y mensajes de error debajo de cada campo. Las operaciones con datos inválidos no se ejecutan.

Exportar/guardar datos y luego importarlos/recuperarlos: Exportación e importación de JSON, copia automática en *localStorage* y reporte CSV.

5. DIAGRAMA UML DE CLASES

El diagrama UML de clases representa de forma gráfica las clases del programa, sus atributos, sus métodos y las relaciones entre ellas. Debe ser coherente con el código implementado, por lo que en esta sección se resumen primero las clases y las relaciones que el diagrama debe mostrar.

5.1. CLASES IDENTIFICADAS

Clases del programa:

Persona, abstracta (base): Define los datos comunes (nombre y teléfono) con sus validaciones y obliga a las clases hijas a implementar *mostrarInformacion()*.

Cliente (derivada de persona): Representa al cliente: identificador, correo y tipo VIP. Calcula su descuento.

Barbero (derivada de persona): Representa al barbero y su especialidad.

Servicio (concreta): Representa un servicio con su precio y calcula el total con descuento.

Cita (concreta): Representa la reserva: cliente, barbero, servicio, fecha, hora, total, notas y estado.

5.2. RELACIONES ENTRE CLASES

Relaciones que debe mostrar el diagrama:

Cliente - *Persona* (herencia): Cliente extiende de Persona (extends, super).

Barbero - *Persona* (herencia): Barbero extiende de Persona (extends, super).

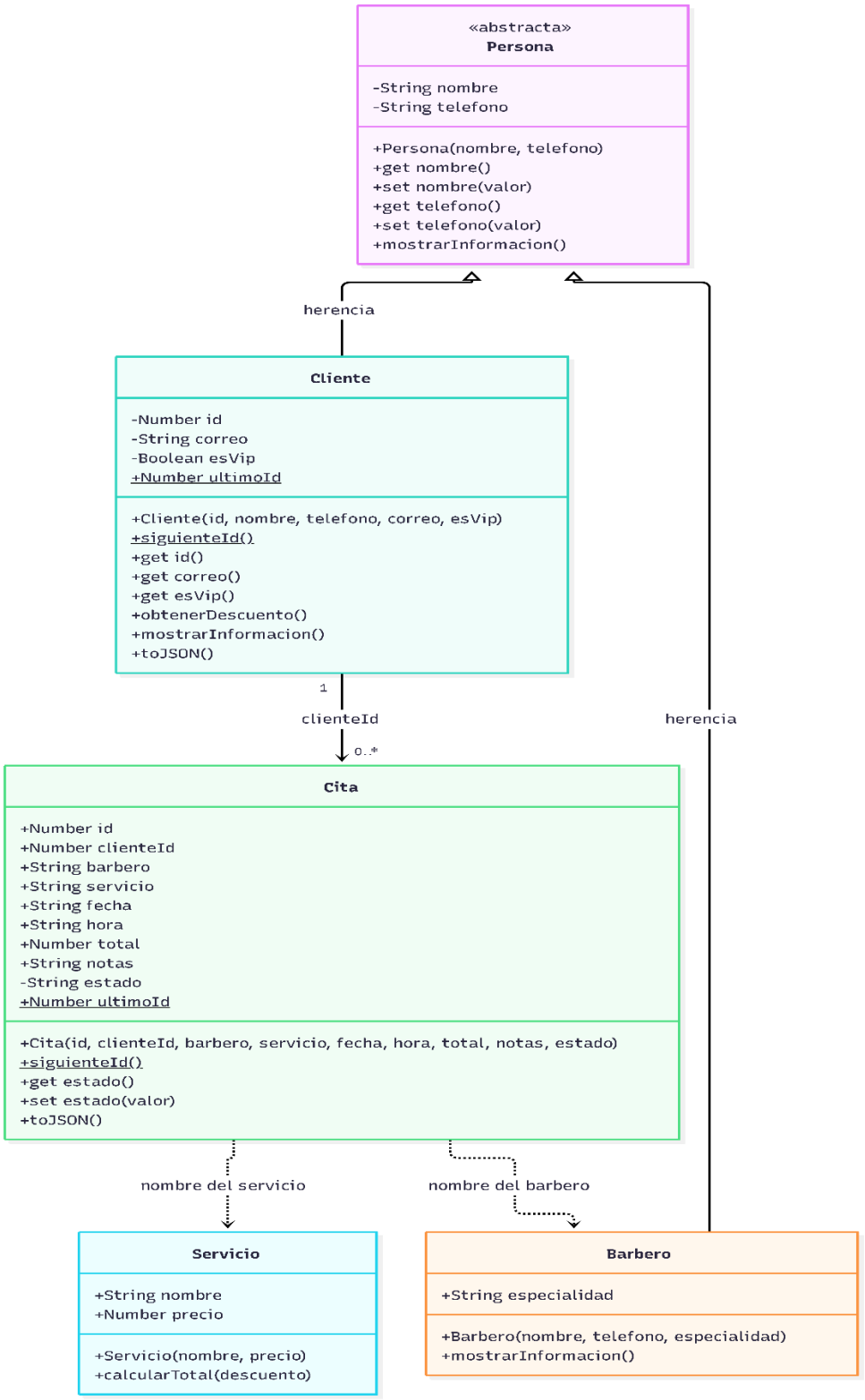
Cita - *Cliente* (asociación): Un cliente puede tener muchas citas (1 a 0..*). La cita guarda el identificador del cliente (*clienteId*).

Cita - *Barbero* (asociación): Un barbero atiende muchas citas (1 a 0..*). La cita guarda el nombre del barbero.

Cita - *Servicio* (asociación): Un servicio puede estar en muchas citas (1 a 0..*). La cita guarda el nombre del servicio.

Servicio - *Cliente* (dependencia de uso): *Servicio.calcularTotal()* recibe el descuento que entrega *Cliente.obtenerDescuento()*.

5.3. DIAGRAMA



Fuente: Elaboración propia.

6. DESCRIPCIÓN TÉCNICA Y PROGRAMACIÓN

Esta sección explica cómo está construido el sistema y cómo se aplican en el código los requisitos técnicos de cada una de las cinco unidades de la asignatura. Los fragmentos de código mostrados son los del archivo *script.js* de la aplicación.

6.1. ORGANIZACIÓN DEL CÓDIGO

El archivo *script.js* está dividido en diez secciones numeradas con comentarios, lo que facilita leerlo, explicarlo y modificarlo durante la defensa. No utiliza frameworks ni bibliotecas externas.

Secciones del archivo *script.js*:

1. Validaciones: Funciones *validarNombre*, *validarTelefono* y *validarCorreo*, que lanzan un error con *throw* si el dato es inválido.

2. Funciones para cadenas: *formatearNombre* y *normalizar*.

3. Clases: *Persona*, *Cliente*, *Barbero*, *Servicio* y *Cita*.

4. Datos: Arreglos *clientes*, *citas*, *barberos* y *servicios*, y el objeto *vistasServicios*.

5. Funciones auxiliares: Búsquedas, fecha actual, *buscarClientes* y *barberoLibre*.

6. Guardar y recuperar: localStorage, *cargarDatos*, exportación JSON y CSV, importación con FileReader.

7. Pantalla: barberos: *pintarBarberos*.

8. Pantalla: clientes: Listado, registro, edición, eliminación y validación en vivo.

9. Pantalla: citas: Reserva, horas libres, total, listado con filtros y cambio de estado.

10. Inicio de la página: *pintarTodo* y conexión de todos los eventos al cargar la página.

6.2. UNIDAD 1: CODIFICACIÓN DE ALGORITMOS

Funciones y parámetros. El programa define e invoca funciones con parámetros para cada tarea: por ejemplo, *barberoLibre(barbero, fecha, hora, idIgnorar)* recibe cuatro parámetros y devuelve verdadero o falso según la disponibilidad del barbero, y *formatearNombre(texto)*

devuelve el nombre normalizado. La operación específica del problema es el control de disponibilidad de los horarios y el cálculo del total con descuento.

Clases y objetos. Se implementan cinco clases con constructores, propiedades y métodos, y se crean objetos con *new*, por ejemplo *new Cliente(id, nombre, telefono, correo, esVip)*.

Comportamiento equivalente a la sobrecarga. JavaScript no admite sobrecarga por firma, por lo que se utilizan parámetros opcionales con valores por defecto y la verificación del tipo y de la presencia de los argumentos:

buscarClientes(criterio): Sin argumento (o vacío): devuelve todos los clientes. Con un número: busca por identificador. Con texto: busca por nombre o teléfono.

new Cliente(id, nombre, telefono, correo = "", esVip = false): Se puede crear con 3 argumentos (cliente normal sin correo) o con los 5 (con correo y tipo VIP).

new Barbero(nombre, telefono, especialidad = "Cortes"): Si no se indica la especialidad, se asigna «Cortes».

servicio.calcularTotal(descuento = 0): Sin argumento devuelve el precio completo; con un descuento (por ejemplo 0.15) devuelve el precio rebajado.

```
// buscarClientes()         -> devuelve todos
// buscarClientes("texto")   -> busca por nombre o teléfono
// buscarClientes(5)         -> busca por ID (número)
function buscarClientes(criterio) {
   if (criterio === undefined || criterio === "") {
      return clientes;
   }
   if (typeof criterio === "number") {
      let encontrado = buscarClientePorId(criterio);
      return encontrado ? [encontrado] : [];
   }
   let texto = normalizar(criterio);
   let resultado = [];
   for (let i = 0; i < clientes.length; i++) {
```

```
    let c = clientes[i];
    if (normalizar(c.nombre).includes(texto) || c.telefono.includes(texto)) {
      resultado.push(c);
    }
  }
  return resultado;
}
```

6.3. UNIDAD 2: TIPOS DE DATOS COMPUESTOS

Arreglos de objetos. Los registros se administran con arreglos de objetos: *clientes* (objetos *Cliente*), *citas* (objetos *Cita*), *barberos* (objetos *Barbero*) y *servicios* (objetos *Servicio*). Se recorren con ciclos *for* y se modifican con métodos como *push* (agregar) y *filter* (eliminar). Además, el objeto *vistasServicios* funciona como un diccionario que asocia el nombre de cada servicio con su imagen y su descripción.

Operaciones sobre cadenas. Se aplican varias operaciones de texto: *trim* para quitar espacios, *toLowerCase* y *charAt/toUpperCase* para dar formato al nombre, *split* y *join* para separar y unir palabras, *includes* para buscar, *padStart* para armar la fecha con dos dígitos y *normalize("NFD")* junto con una expresión regular para quitar las tildes y poder comparar «José» con «jose».

```
// "  juan PEREZ " -> "Juan Perez"
function formatearNombre(texto) {
  let palabras = texto.trim().toLowerCase().split(" ");
  let resultado = [];
  for (let i = 0; i < palabras.length; i++) {
    if (palabras[i] !== "") {
      resultado.push(palabras[i].charAt(0).toUpperCase() + palabras[i].slice(1));
    }
  }
  return resultado.join(" ");
}
// Quita tildes y pasa a minúsculas para buscar: "José" -> "jose"
function normalizar(texto) {
```

```
  return texto.toLowerCase().normalize("NFD").replace(/[̀-ͯ]/g, "");
}
```

Entidades como objetos. Cada entidad del problema (cliente, barbero, servicio y cita) se representa mediante un objeto de su clase, con sus propiedades organizadas, lo que permite recorrer, buscar y mostrar la información de forma ordenada.

6.4. UNIDAD 3: ARCHIVOS DE TEXTO

Alternativa elegida. Para guardar y recuperar los registros se eligió trabajar desde el navegador, usando archivos JSON, en lugar de Node.js con el módulo *fs*. La razón es que la consigna exige que la aplicación se abra y se ejecute con solo abrir los archivos HTML; usar Node.js obligaría a instalar un entorno y a levantar un programa adicional. Con la solución elegida basta con un navegador.

La persistencia se resuelve con tres mecanismos complementarios:

- Copia automática (localStorage): después de cada cambio se guarda el estado completo con *guardarEnNavegador()*, y al abrir la página se recupera con *cargarDatos()*. Así los datos sobreviven al cerrar y volver a abrir la aplicación.
- Exportar JSON: *exportarJSON()* genera el archivo *peluqueria.json* con los clientes y las citas, que se descarga con la función *descargar()* (objeto *Blob* y enlace temporal).
- Importar JSON: *importarJSON()* lee el archivo elegido con *FileReader* y reconstruye los objetos reales de las clases *Cliente* y *Cita* mediante *cargarDatos()*.

Como los atributos privados no se convierten automáticamente a JSON, las clases *Cliente* y *Cita* definen el método *toJSON()*, que indica exactamente qué datos se guardan. Al importar, *cargarDatos()* valida la estructura del archivo y vuelve a crear los objetos con sus constructores, de modo que los datos importados pasan por las mismas validaciones que los ingresados a mano.

```
// Convierte un texto JSON en objetos reales (Cliente y Cita)
function cargarDatos(texto) {
  let datos = JSON.parse(texto);   // si el JSON está mal, lanza error
  if (!Array.isArray(datos.clientes) || !Array.isArray(datos.citas)) {
    throw new Error("El archivo no tiene clientes y citas");
  }
```

```
    let nuevosClientes = [];
    let nuevasCitas = [];
    Cliente.ultimoId = 0;
    Cita.ultimoId = 0;
    for (let i = 0; i < datos.clientes.length; i++) {
        let d = datos.clientes[i];
        nuevosClientes.push(new Cliente(d.id, d.nombre, d.telefono, d.correo, d.esVip));
    }
    for (let i = 0; i < datos.citas.length; i++) {
        let d = datos.citas[i];
        nuevasCitas.push(new Cita(d.id, d.clienteId, d.barbero, d.servicio, d.fecha, d.hora, d.total,
d.notas, d.estado));
    }
    clientes = nuevosClientes;
    citas = nuevasCitas;
}
```

6.5. UNIDAD 4: PROGRAMACIÓN ORIENTADA A OBJETOS

6.5.1. Encapsulamiento

Los atributos sensibles se declaran privados con el prefijo # (por ejemplo *#nombre*, *#telefono*, *#id*, *#correo*, *#esVip* y *#estado*) y solo se accede a ellos mediante métodos de acceso (*get* y *set*). Los *set* incluyen validaciones: por ejemplo, el setter del teléfono llama a *validarTelefono* y el setter del estado de la cita solo acepta «Pendiente», «Completada» o «Cancelada».

6.5.2. Abstracción

La clase base Persona es abstracta: si alguien intenta crearla directamente con *new Persona()*, el constructor lanza un error gracias a la verificación *new.target === Persona*. Además, su método *mostrarInformacion()* actúa como contrato: lanza un error si una clase hija no lo implementa, obligando a sobrescribirlo. Así se representa una clase abstracta mediante convenciones y verificaciones, como indica la consigna.

```
// ----- Clase base ABSTRACTA -----
// No se puede crear con new Persona(), solo sirve para heredar
class Persona {
    // atributos privados (encapsulamiento)
    #nombre;
    #telefono;
    constructor(nombre, telefono) {
        if (new.target === Persona) {
            throw new Error("Persona es una clase abstracta, no se puede instanciar");
        }
        this.nombre = nombre;
        this.telefono = telefono;
    }
    // getters y setters (métodos de acceso)
    get nombre() {
        return this.#nombre;
    }
    set nombre(valor) {
        validarNombre(valor);
        this.#nombre = formatearNombre(valor);
    }
    get telefono() {
        return this.#telefono;
    }
    set telefono(valor) {
        validarTelefono(valor);
        this.#telefono = valor;
    }
    // método abstracto: las clases hijas DEBEN sobrescribirlo
    mostrarInformacion() {
        throw new Error("Este método debe implementarse en la clase hija");
```

```
    }
}
```

6.5.3. Herencia y polimorfismo

Las clases Cliente y Barbero heredan de Persona con *extends* y llaman al constructor de la clase base con *super(nombre, telefono)*. Ambas sobrescriben el método *mostrarInformacion()* con un resultado distinto: el cliente muestra su tipo (Normal o VIP) y el barbero su especialidad. Esto es polimorfismo: el mismo método responde de manera diferente según la clase del objeto.

```
//Clase derivada 2: Barbero
class Barbero extends Persona {
    constructor(nombre, telefono, especialidad = "Cortes") {
        super(nombre, telefono);
        this.especialidad = especialidad;
    }
// sobrescribe el método de Persona (polimorfismo)
    mostrarInformacion() {
        return this.nombre + " - " + this.telefono + " (" + this.especialidad + ")";
    }
}
```

6.5.4. Miembros estáticos y métodos de instancia

Las clases *Cliente* y *Cita* tienen un atributo estático *ultimoId* y un método estático *siguienteId()* que entregan el siguiente identificador disponible sin necesidad de crear un objeto. El resto de los métodos (*obtenerDescuento(), calcularTotal(), mostrarInformacion(), toJSON()*) son métodos de instancia.

6.6. DESCRIPCIÓN DE CLASES, ATRIBUTOS Y MÉTODOS

Clase *Persona* (abstracta):

#nombre (privado): Nombre de la persona (se guarda con formato correcto).

#telefono (privado): Teléfono de 8 dígitos que empieza con 6 o 7.

constructor(nombre, telefono) (público): Impide instanciar la clase directamente; asigna los datos mediante los setters.

get/set nombre (público): Lee el nombre; al asignar valida y le da formato.

get/set telefono (público): Lee el teléfono; al asignar valida el formato.

mostrarInformacion(), público (abstracto): Lanza un error; debe ser sobrescrito por las clases hijas.

Clase *Cliente* (hereda de *Persona*):

#id (privado): Identificador único del cliente.

#correo (privado): Correo electrónico (opcional, con formato válido).

#esVip (privado): Indica si el cliente es VIP.

ultimoId (estático) (público): Último identificador asignado.

siguienteId() (estático) (público): Devuelve el siguiente identificador disponible.

constructor(id, nombre, telefono, correo = "", esVip = false) (público): Llama a super, valida el correo y asigna los datos.

get id / correo / esVip (público): Métodos de acceso de solo lectura.

obtenerDescuento() (público): Devuelve 0.15 si el cliente es VIP y 0 en caso contrario.

mostrarInformacion() (público): Sobrescribe el método de Persona: nombre, teléfono y tipo.

toJSON() (público): Devuelve los datos que se guardan en el archivo.

Clase *Barbero* (hereda de *Persona*):

especialidad (público): Especialidad del barbero.

constructor(nombre, telefono, especialidad = "Cortes") (público): Llama a super y asigna la especialidad.

mostrarInformacion() (público): Sobrescribe el método de Persona: nombre, teléfono y especialidad.

Clase *Servicio*:

nombre (público): Nombre del servicio.

precio (público): Precio en bolivianos.

constructor(nombre, precio) (público): Crea el servicio.

calcularTotal(descuento = 0) (público): Devuelve el precio menos el descuento indicado.

Clase *Cita*:

#estado (privado): Estado de la cita: Pendiente, Completada o Cancelada.

id, clienteId, barbero, servicio, fecha, hora, total, notas (público): Datos de la reserva.

ultimoId / siguienteId() (estáticos) (público): Control de los identificadores de las citas.

constructor(id, clienteId, barbero, servicio, fecha, hora, total, notas = "", estado = "Pendiente") (público): Crea la cita; las notas y el estado son opcionales.

get/set estado (público): Lee el estado; al asignar rechaza valores no válidos.

toJSON() (público): Devuelve los datos que se guardan en el archivo.

6.7. UNIDAD 5: VALIDACIÓN DE APLICACIONES

La validación se realiza en dos niveles que se complementan: las validaciones intrínsecas de HTML, que el navegador aplica al enviar el formulario, y las validaciones de JavaScript, que muestran mensajes claros y se aplican también al importar archivos.

Validaciones por campo:

Nombre: en HTML, *required, minlength=3, maxlength=60*; en JavaScript, obligatorio, mínimo 3 caracteres, solo letras y espacios (acepta tildes y ñ).

Teléfono: en HTML, *required, pattern=[67][0-9]{7}, maxlength=8*; en JavaScript, obligatorio, 8 dígitos y empieza con 6 o 7; solo acepta números mientras se escribe; no se repite entre clientes.

Correo: en HTML, *type=email*; en JavaScript, opcional; si se escribe, debe tener formato de correo válido.

Cliente, barbero, servicio: en HTML, *required en los selectores*; en JavaScript, se verifica que el cliente y el servicio existan.

Fecha: en HTML, *type=date, required, min = fecha de hoy*; en JavaScript, no se permite reservar en una fecha pasada.

Hora: en HTML, *required*; en JavaScript, solo se ofrecen horas libres; se vuelve a verificar la disponibilidad al guardar.

Notas: en HTML, *maxlength=100*; en JavaScript, opcional.

Eventos de validación. Se utilizan varios eventos: *input* (validación del nombre y del teléfono mientras la persona escribe, y búsquedas en tiempo real), *blur* (validación del correo al salir del campo), *change* (actualización de horas, total e imagen al cambiar barbero, fecha, cliente o servicio) y *submit* (guardado de clientes y de citas).

Manejo de errores. Las funciones de validación lanzan errores con *throw new Error(...)*. Los formularios capturan esos errores con *try/catch* y muestran el mensaje en rojo, y el bloque *finally* se ejecuta siempre para guardar una copia y actualizar la pantalla. La importación de archivos también usa *try/catch/finally*, por lo que un archivo dañado no detiene la aplicación.

```
function guardarCliente(evento) {
  evento.preventDefault();   // evita que la página se recargue
  let nombre = document.getElementById("nombre").value;
  let telefono = document.getElementById("telefono").value.trim();
  let correo = document.getElementById("correo").value.trim();
  let esVip = document.getElementById("tipo").value === "vip";
  try {
    // no permitir teléfonos repetidos
    for (let i = 0; i < clientes.length; i++) {
      if (clientes[i].telefono === telefono && clientes[i].id !== clienteEditando) {
        throw new Error("Ya existe un cliente con ese teléfono");
      }
    }
```

```
        if (clienteEditando === null) {
            // REGISTRAR
            let nuevo = new Cliente(Cliente.siguienteId(), nombre, telefono, correo, esVip);
            clientes.push(nuevo);
            mostrarMensaje("Cliente registrado correctamente", "green");
        } else {
        // MODIFICAR: se reemplaza el objeto en la misma posición
            for (let i = 0; i < clientes.length; i++) {
                if (clientes[i].id === clienteEditando) {
                    clientes[i] = new Cliente(clienteEditando, nombre, telefono, correo, esVip);
                }
            }
            mostrarMensaje("Cliente modificado correctamente", "green");
        }
        cancelarCliente();
    } catch (error) {
        mostrarMensaje(error.message, "red");
    } finally {
        guardarEnNavegador();   // se ejecuta siempre
        pintarTodo();
    }
}
// Validación en vivo: muestra el error debajo del campo
function validarEnVivo(idCampo, idError, funcionValidar) {
    let valor = document.getElementById(idCampo).value;
    try {
        funcionValidar(valor);
        document.getElementById(idError).textContent = "";
    } catch (error) {
        document.getElementById(idError).textContent = error.message;
    }
```

```
}
```

6.8. OPERACIONES DE NEGOCIO

La principal operación de negocio es el control de disponibilidad. La función *barberoLibre()* recorre las citas y devuelve falso si el barbero ya tiene una cita no cancelada en esa fecha y hora (ignorando la cita que se está editando). Esta función se usa dos veces: para mostrar solo las horas libres en el selector y, por seguridad, otra vez al guardar la cita.

```
// Operación de negocio: ¿el barbero está libre en esa fecha y hora?
function barberoLibre(barbero, fecha, hora, idIgnorar) {
  for (let i = 0; i < citas.length; i++) {
    let c = citas[i];
    if (c.id !== idIgnorar && c.barbero === barbero && c.fecha === fecha &&
      c.hora === hora && c.estado !== "Cancelada") {
      return false;
    }
  }
  return true;
}
```

Otras operaciones de negocio son el cálculo del total (*servicio.calcularTotal(cliente.obtenerDescuento())*), la suma de los ingresos cobrados (solo citas completadas), el bloqueo de la eliminación de clientes que tienen citas y el control del estado de cada cita.

7. PRUEBAS Y RESULTADOS

7.1. ENTORNO Y METODOLOGÍA

Las pruebas se realizaron abriendo el archivo *index.html* en un navegador, usando como base los datos del archivo *datosejemplo.json*. Para cada caso se registró el dato de entrada, el resultado esperado y el resultado obtenido. En todos los casos el resultado obtenido coincidió con el esperado.

7.2. PRUEBAS DE CLIENTES

Tabla 3

Pruebas del módulo de clientes

N.º	Prueba y dato de entrada	Resultado esperado	Resultado obtenido
P01	Nombre con número: «Ju4n»	Mensaje de error debajo del campo	Muestra «El nombre solo puede tener letras»
P02	Teléfono incorrecto: «12345»	Mensaje de error	Muestra «El teléfono debe tener 8 dígitos y empezar con 6 o 7»
P03	Correo incorrecto: «correo-malo»	Mensaje de error al salir del campo	Muestra «El correo no es válido»
P04	Registrar « juan PEREZ », 76543210, VIP	Cliente guardado con formato correcto	Se guarda como «Juan Perez» (VIP): «Cliente registrado correctamente»
P05	Registrar otro cliente con el teléfono 76543210	Se rechaza el registro	Muestra «Ya existe un cliente con ese teléfono»
P06	Buscar «jose» en clientes	Encuentra a «José Rojas» (sin distinguir tildes)	La lista muestra solo a «José Rojas»
P07	Editar cliente 2: nombre «José Rojas Vaca»	La lista se actualiza	El nombre cambia en la lista de clientes
P08	Eliminar a un cliente que tiene citas	Se rechaza la eliminación	Muestra «No se puede eliminar: el cliente tiene citas» ✔

Fuente: Elaboración propia.

Figura 11

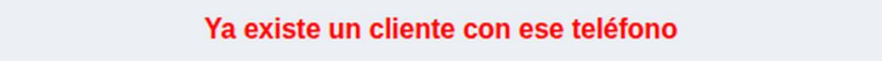
Mensaje de éxito al registrar un cliente (P04)



Fuente: Elaboración propia.

Figura 12

Mensaje de error al repetir el teléfono (P05)



Fuente: Elaboración propia.

Figura 13

Mensaje de error al intentar eliminar un cliente con citas (P08)



Fuente: Elaboración propia.

7.3. PRUEBAS DE CITAS

Tabla 4

Pruebas del módulo de citas

N.º	Prueba y dato de entrada	Resultado esperado	Resultado obtenido
P09	Reservar: Carlos Mamani (VIP), Luis Velasquez, High Fade (Bs 55), 28/10/2026, 11:00	Total con 15 % de descuento: Bs 46.75	Muestra «Cita reservada. Total: Bs 46.75»
P10	Ver las horas libres de Luis Velasquez el 28/10/2026 después de reservar	La hora 11:00 ya no aparece	La lista pasa de 14 a 13 horas, sin las 11:00
P11	Completar la cita 1 (Taper Fade, Bs 42.50) y filtrar por “Completada”	Ingresos = 42.50 + 25.00	Muestra “Ingresos cobrados: Bs 67.50”
P12	Filtrar por estado “Completada”	Solo citas completadas	Se muestran 2 citas (ID 1 y 4)
P13	Escribir “charles” en la búsqueda de citas	Solo las citas de ese barbero	Se muestra solo la cita de Charles Cedeño

Fuente: Elaboración propia.

Figura 14

Mensaje de éxito al reservar una cita con total y descuento VIP



Fuente: Elaboración propia.

Horas libres de Luis Velasquez el 28/10/2026:

Antes de reservar: 09:00, 10:00, 11:00, 12:00, 13:00, 14:00, 15:00, 16:00, 17:00, 18:00, 19:00, 20:00, 21:00, 22:00 (14 horas).

Después de reservar: 09:00, 10:00, 12:00, 13:00, 14:00, 15:00, 16:00, 17:00, 18:00, 19:00, 20:00, 21:00, 22:00 (13 horas).

7.4. PRUEBAS DE ARCHIVOS

Tabla 5

Pruebas de guardado y recuperación

N.º	Prueba y dato de entrada	Resultado esperado	Resultado obtenido
P14	Importar datosejemplo.json	Se cargan los datos de ejemplo	Aparecen 3 clientes y 4 citas
P15	Exportar JSON	Se descarga peluqueria.json	Archivo con los clientes y las citas actuales
P16	Exportar reporte CSV	Se descarga reporte_citas.csv	Archivo con encabezado y una fila por cita, abre bien con tildes
P17	Recargar la página	Los datos se conservan	Los clientes y las citas siguen en pantalla
P18	Importar un JSON inválido: {"clientes": 5}	Error controlado, sin cerrar la aplicación	Muestra «Error al importar: El archivo no tiene clientes y citas»

Fuente: Elaboración propia.

Figura 15

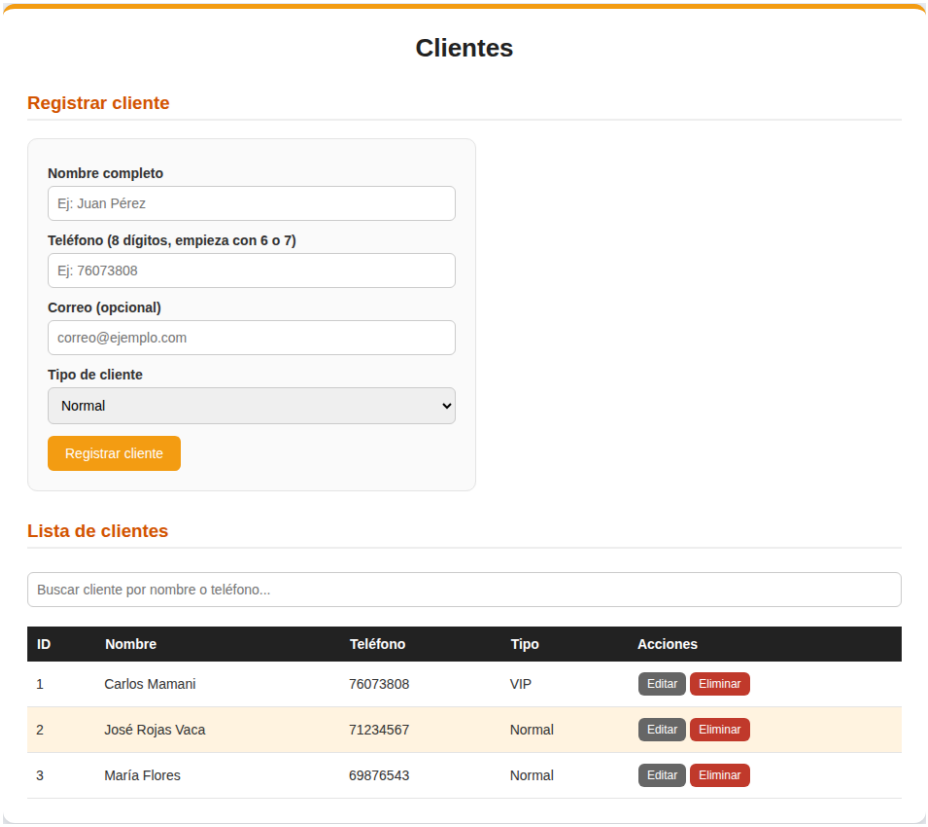
Mensaje de error controlado al importar un archivo inválido



Fuente: Elaboración propia.

Figura 16

Datos conservados después de recargar la página



Fuente: Elaboración propia.

7.5. RESUMEN DE RESULTADOS

Tabla 6

Resumen de las pruebas

Módulo	Pruebas	Correctas	Fallidas
Clientes	8	8	0
Citas	5	5	0
Archivos	5	5	0
Total	18	18	0

Fuente: Elaboración propia.

Los resultados muestran que el sistema valida los datos, evita los cruces de horario, calcula correctamente los totales y conserva la información entre ejecuciones, cumpliendo con las funcionalidades mínimas exigidas.

8. CONCLUSIONES

Se logró desarrollar una aplicación web funcional para la Barbería Zurdo que resuelve el problema planteado: permite registrar clientes, reservar citas sin cruces de horario, calcular automáticamente el total con descuento VIP y conservar la información entre ejecuciones mediante archivos. El sistema se ejecuta abriendo un solo archivo en el navegador, sin base de datos ni frameworks, tal como pedía la consigna.

El problema se resolvió identificando primero las entidades del negocio y sus reglas, y luego traduciéndolas a clases y funciones. La regla más importante, que un barbero no pueda tener dos citas a la misma hora, se implementó con la función *barberoLibre()*, y se aplica tanto al mostrar las horas disponibles como al guardar la cita. Las validaciones en HTML y JavaScript impiden que ingresen datos inválidos, y el manejo de errores con *try/catch/finally* evita que la aplicación se detenga ante un archivo dañado.

Como trabajo futuro, el sistema podría ampliarse permitiendo editar el catálogo de barberos y servicios desde la pantalla, agregando inicio de sesión para el personal, un reporte de ingresos por barbero o por fecha y la impresión de comprobantes. El proyecto también permitió comprender que organizar bien el código en clases y funciones pequeñas facilita las pruebas, la corrección de errores y la explicación del programa.

9. BIBLIOGRAFÍA Y FUENTES CONSULTADAS

MDN Web Docs. (s.f.). *Blob*. Obtenido de https://developer.mozilla.org/es/docs/Web/API/Blob

MDN Web Docs. (s.f.). *Clases (JavaScript)*. Obtenido de https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes

MDN Web Docs. (s.f.). *FileReader*. Obtenido de https://developer.mozilla.org/es/docs/Web/API/FileReader

MDN Web Docs. (s.f.). *String.prototype.normalize()*. Obtenido de https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/String/normalize

MDN Web Docs. (s.f.). *Window: propiedad localStorage*. Obtenido de https://developer.mozilla.org/es/docs/Web/API/Window/localStorage

Shafranovich, Y.. (2005). *Common Format and MIME Type for Comma-Separated Values (CSV) Files (RFC 4180)*. Obtenido de https://www.rfc-editor.org/rfc/rfc4180

10. INVESTIGACIÓN E INNOVACIÓN

10.1. NECESIDAD ADICIONAL QUE RESUELVE

Con los requisitos básicos, la barbería podía registrar y consultar citas, pero seguían existiendo tres necesidades: llevar la información a otras herramientas, por ejemplo, para revisar los ingresos en una hoja de cálculo; recuperar los datos desde un archivo guardado en otra computadora o en otra ejecución, sin depender de la copia del navegador; y encontrar rápidamente una cita cuando la lista crece, combinando criterios de búsqueda. Además, se detectó que ayudaba al cliente y al barbero ver una imagen de referencia del servicio antes de confirmar la reserva.

10.2. DESCRIPCIÓN DE LA MEJORA

Mejoras implementadas:

1. Reporte CSV de citas: Botón “Exportar reporte CSV” que genera el archivo *reporte_citas.csv* con el ID, cliente, barbero, servicio, fecha, hora, total y estado de cada cita, listo para abrir en Excel.

2. Lectura de archivos con FileReader: Botón “Importar JSON” que lee un archivo elegido por el usuario y recupera los clientes y las citas, validando su estructura.

3. Filtros combinados: La lista de citas se filtra a la vez por texto (cliente, barbero o servicio) y por estado, y recalcula el resumen de ingresos.

4. Imagen de referencia por servicio: Al elegir un servicio en la reserva se muestra su imagen y su nombre; sin selección se muestra una imagen de la barbería.

10.3. FUENTES CONSULTADAS

Fuentes utilizadas para la investigación:

MDN Web Docs: FileReader (developer.mozilla.org/es/docs/Web/API/FileReader): Uso de *readAsText* y del evento *onload* para leer el archivo.

MDN Web Docs: Blob (developer.mozilla.org/es/docs/Web/API/Blob): Creación del archivo en memoria para la descarga.

RFC 4180: formato CSV (rfc-editor.org/rfc/rfc4180): Estructura del archivo CSV: encabezado y filas separadas por comas.

MDN Web Docs: String.normalize() (developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/String/normalize): Búsqueda sin tildes en los filtros.

10.4. ALTERNATIVAS CONSIDERADAS

Comparación de alternativas:

Exportar reporte: Archivo Excel (.xlsx) con una biblioteca externa. Desventaja: requiere agregar una biblioteca y más tiempo.

Leer archivos: Leer con fetch() desde la carpeta. Desventaja: los navegadores lo bloquean al abrir el HTML desde el disco.

Buscar citas: Un solo filtro de texto. Desventaja: poco preciso cuando hay muchas citas.

10.5. JUSTIFICACIÓN DE LA ELECCIÓN

Las soluciones elegidas son las más viables para una semana de trabajo, mantienen a JavaScript como lenguaje principal, no requieren servicios de pago, bases de datos ni bibliotecas externas, y se relacionan directamente con la problemática: la barbería necesita sacar sus citas a Excel, recuperar sus datos desde un archivo y ubicar rápidamente una cita. Además, funcionan al abrir el HTML directamente en el navegador.

10.6. IMPLEMENTACIÓN

Reporte CSV. La función *exportarCSV()* arma un texto con el encabezado y una línea por cada cita, buscando el nombre del cliente a partir de su identificador (si el cliente fue eliminado,

escribe (eliminado)). Al principio del texto se agrega el carácter BOM de UTF-8 para que Excel muestre bien las tildes, y la descarga se hace con la función *descargar()*, que crea un *Blob* y un enlace temporal.

```
// INNOVACIÓN: reporte en CSV (se abre con Excel)
function exportarCSV() {
   let texto = "ID,Cliente,Barbero,Servicio,Fecha,Hora,Total,Estado\n";
   for (let i = 0; i < citas.length; i++) {
      let c = citas[i];
      let cliente = buscarClientePorId(c.clienteId);
      let nombre = cliente ? cliente.nombre : "(eliminado)";
      texto += c.id + "," + nombre + "," + c.barbero + "," + c.servicio + "," +
            c.fecha + "," + c.hora + "," + c.total + "," + c.estado + "\n";
   }
   descargar("reporte_citas.csv", "" + texto, "text/csv");
}
```

Lectura con FileReader. La función *importarJSON()* toma el archivo elegido, lo lee como texto y, cuando termina la lectura (evento *onload*), llama a *cargarDatos()* dentro de un bloque *try/catch/finally*: si el archivo es válido muestra un mensaje de éxito; si no, muestra el error; y en ambos casos actualiza la pantalla.

```
// INNOVACIÓN: leer un archivo con FileReader
function importarJSON(evento) {
   let archivo = evento.target.files[0];
   if (!archivo) {
      return;
   }
   let lector = new FileReader();
   lector.onload = function () {
      try {
         cargarDatos(lector.result);
         guardarEnNavegador();
         mostrarMensaje("Datos importados correctamente", "green");
```

```
        } catch (error) {
            mostrarMensaje("Error al importar: " + error.message, "red");
        } finally {
            pintarTodo();
        }
    };
    lector.readAsText(archivo);
    evento.target.value = "";
}
```

Filtros combinados e imagen por servicio. La función *pintarCitas()* recorre las citas y solo muestra las que cumplen a la vez la búsqueda de texto y el estado elegido, y calcula el resumen de ingresos con las citas completadas que quedan visibles. La función *actualizarVistaServicio()* consulta el objeto *vistasServicios* para cambiar la imagen y el nombre mostrados.

10.7. EVIDENCIAS DE FUNCIONAMIENTO

Tabla 7

Contenido del archivo reporte_citas.csv generado

ID	Cliente	Barbero	Servicio	Fecha	Hora	Total	Estado
1	Carlos Mamani	Luis Velasquez	Taper Fade	2026-10-26	10:00	42.5	Completada
2	José Rojas Vaca	Charles Cedeño	Mullet	2026-10-26	10:00	55	Pendiente
3	María Flores	Oscar Perez	Tinte	2026-10-27	15:00	115	Pendiente
4	José Rojas Vaca	Josep Cazas	Barba	2026-10-22	18:00	25	Completada
5	Carlos Mamani	Luis Velasquez	High Fade	2026-10-28	11:00	46.75	Pendiente

Fuente: Elaboración propia.

ANEXOS

ANEXO A: INSTRUCCIONES DE EJECUCIÓN

1. Descomprimir la carpeta del proyecto, manteniendo juntos los archivos *index.html*, *estilos.css*, *script.js*, *datosejemplo.json* y la carpeta *imagenes*.

2. Abrir el archivo *index.html* con un navegador (Google Chrome, Microsoft Edge o Firefox). No requiere instalación ni conexión a internet.
3. Para cargar los datos de ejemplo, ir a la sección «Guardar y recuperar datos», pulsar Importar JSON y elegir *datosejemplo.json*.
4. Para guardar la información en un archivo, pulsar Exportar JSON. Para obtener el reporte de citas para Excel, pulsar Exportar reporte CSV.
5. Para recuperar la información en otra ejecución, volver a pulsar Importar JSON y elegir el archivo guardado. Además, la aplicación conserva automáticamente los datos en el navegador.

ANEXO B: ARCHIVO DE DATOS DE EJEMPLO

Estructura del archivo *datosejemplo.json*, con un arreglo de clientes y un arreglo de citas:

{

"clientes": [

{ "id": 1, "nombre": "Carlos Mamani", "telefono": "76073808", "correo": "carlos@mail.com", "esVip": true },

{ "id": 2, "nombre": "José Rojas", "telefono": "71234567", "correo": "", "esVip": false },

{ "id": 3, "nombre": "María Flores", "telefono": "69876543", "correo": "maria@mail.com", "esVip": false }

],

"citas": [

{ "id": 1, "clienteId": 1, "barbero": "Luis Velasquez", "servicio": "Taper Fade", "fecha": "2026-10-26", "hora": "10:00", "total": 42.5, "notas": "", "estado": "Pendiente" },

{ "id": 2, "clienteId": 2, "barbero": "Charles Cedeño", "servicio": "Mullet", "fecha": "2026-10-26", "hora": "10:00", "total": 55, "notas": "Foto de referencia", "estado": "Pendiente" },

{ "id": 3, "clienteId": 3, "barbero": "Oscar Perez", "servicio": "Tinte", "fecha": "2026-10-27", "hora": "15:00", "total": 115, "notas": "", "estado": "Pendiente" },

{ "id": 4, "clienteId": 2, "barbero": "Josep Cazas", "servicio": "Barba", "fecha": "2026-10-22", "hora": "18:00", "total": 25, "notas": "", "estado": "Completada" }

]

}

ANEXO C: CÓDIGO DE LAS CLASES

Código completo de las clases *Cliente*, *Servicio* y *Cita* del archivo *script.js*

```
// Clase derivada 1: Cliente
class Cliente extends Persona {
    #id;
    #correo;
    #esVip;

// miembro estático
    static ultimoId = 0;
    static siguienteId() {
        return Cliente.ultimoId + 1;
    }
// "Sobrecarga": correo y esVip son parámetros opcionales
    constructor(id, nombre, telefono, correo = "", esVip = false) {
        super(nombre, telefono);
        validarCorreo(correo);
        this.#id = id;
        this.#correo = correo;
        this.#esVip = esVip;
        if (id > Cliente.ultimoId) {
            Cliente.ultimoId = id;
        }
    }
    get id() { return this.#id; }
    get correo() { return this.#correo; }
    get esVip() { return this.#esVip; }
    // el cliente VIP tiene 15% de descuento
    obtenerDescuento() {
        if (this.#esVip) {
            return 0.15;
```

```
        }
        return 0;
    }
    // sobrescribe el método de Persona (polimorfismo)
    mostrarInformacion() {
        let tipo = this.#esVip ? "VIP" : "Normal";
        return this.nombre + " - " + this.telefono + " (" + tipo + ")";
    }
    // para poder guardar en JSON (los campos privados no se guardan solos)
    toJSON() {
        return {
            id: this.#id,
            nombre: this.nombre,
            telefono: this.telefono,
            correo: this.#correo,
            esVip: this.#esVip
        };
    }
}
// Servicio
class Servicio {
    constructor(nombre, precio) {
        this.nombre = nombre;
        this.precio = precio;
    }
// calcula el precio con descuento
    calcularTotal(descuento = 0) {
        return this.precio - this.precio * descuento;
    }
}
// Cita
```

```
class Cita {
    #estado;
    static ultimoId = 0;
    static siguienteId() {
        return Cita.ultimoId + 1;
    }
    constructor(id, clienteId, barbero, servicio, fecha, hora, total, notas = "", estado =
"Pendiente") {
        this.id = id;
        this.clienteId = clienteId;
        this.barbero = barbero;
        this.servicio = servicio;
        this.fecha = fecha;
        this.hora = hora;
        this.total = total;
        this.notas = notas;
        this.estado = estado;
        if (id > Cita.ultimoId) {
            Cita.ultimoId = id;
        }
    }
    get estado() {
        return this.#estado;
    }
    set estado(valor) {
        if (valor !== "Pendiente" && valor !== "Completada" && valor !== "Cancelada") {
            throw new Error("Estado no válido");
        }
        this.#estado = valor;
    }
    toJSON() {
```

```
        return {
            id: this.id,
            clienteId: this.clienteId,
            barbero: this.barbero,
            servicio: this.servicio,
            fecha: this.fecha,
            hora: this.hora,
            total: this.total,
            notas: this.notas,
            estado: this.estado
        };
    }
}
```